ENGINEERING NOTEBOOK

Assay

An ETL pipeline, and which data-quality check catches which defect

Author Nikhil Sinha

Repository github.com/Nikhil201716/Assay-Data-Quality

Stack Python · standard library ETL · SQLite · pytest + Hypothesis · GE-style checks

Edition 1.0

How to read this book

This is a data-quality-testing notebook that happens to be about a retail-orders pipeline. It is written for someone building or testing data pipelines who wants to know, concretely, which check catches which class of defect -- and which corruption escapes them all.

Every listing is extracted from the repository at build time and every number is read from the reports a real run wrote, so nothing here can drift from the code.

Contents

1. A green ETL job proves nothing

1.1 The finding

1.2 Complementary, not ranked

2. Why 'the pipeline ran' is the trap

2.1 Success is not quality

3. The pipeline

3.1 Generation

3.2 Clean data satisfies every rule

4. The source and warehouse schemas

4.1 The orders source

4.2 The warehouse table

5. Products and customers

5.1 The catalog

6. The E in ETL: extraction hazards

6.1 Where bad data comes from

7. The transform

7.1 Pure helpers

7.2 The mechanical join

8. The load

8.1 Replace, not append

9. Orchestrating the run

9.1 The runner

10. The check suite

10.1 The mechanisms

11. Profiling before expecting

11.1 The profile is the source of the rules

12. Schema checks

12.1 Structure only

13. The expectation suite

13.1 Great-Expectations, hand-rolled

14. Referential integrity

14.1 Keys that point nowhere

15. Reconciliation

15.1 Against the source of truth

16. Distribution checks

16.1 Mean and spread vs a baseline

17. Choosing thresholds

17.1 Too tight cries wolf; too loose misses drift

18. When a distribution check becomes anomaly detection

18.1 Fixed bands, then learned ones

19. The dimensions of data quality

19.1 A vocabulary, mapped to checks

20. The defect catalog

20.1 Data and process defects

21. How each defect is injected

21.1 Data defects

21.2 A process defect

22. The scoring harness

22.1 One defect, one run

23. Turning the matrix into the finding

23.1 Who catches what the others miss

24. Results

24.1 The catch matrix

24.2 By defect class

24.3 The gaps

25. Tracing four defects

25.1 D10 silent drop: only reconciliation

25.2 D03 orphan: only referential

25.3 D11 scale: distribution (and reconciliation), not expectations

25.4 D13 valid-but-wrong: nothing

26. Property-based testing of the transforms

26.1 Properties, not examples

27. Testing the checks

27.1 Meta-tests

28. More of the test suite

28.1 Determinism and date properties

28.2 Clean passes; additive is not flagged

29. Pinning the remaining cells

29.1 Each unique cell is a test

30. Precision: not flagging the safe change

30.1 The additive probe

31. Design decisions

31.1 Why a mechanical transform

31.2 Why hand-rolled checks, not Great Expectations

31.3 Why the standard library only

32. Where the checks run: the quality gate

32.1 A check is a decision, not just an assertion

33. Severity, and quarantine

33.1 Reject, warn, or quarantine

34. From a red check to a green pipeline

34.1 What you do when a check fires

35. The warehouse, and reconciliation as SQL

35.1 Read-back and aggregates

36. Checks at scale

36.1 Sampling, push-down, and cost

37. Data observability

37.1 The five pillars

38. From here to Great Expectations, dbt and Soda

38.1 The same concepts, at scale

39. Data contracts

39.1 Shift the guarantee upstream

40. Quality in production

40.1 From batch checks to monitors

41. Incremental loads and late data

41.1 Why full-refresh hides problems

42. Lineage and provenance

42.1 Detection is not enough without lineage

43. A single data-quality score, and why to distrust it

43.1 The percentage that hides the gap

44. What this does not establish

44.1 A curated catalog

44.2 'Caught' means one check fired

45. The finding, restated

45.1 Everything in one place

45.2 The paragraph

46. Where to read further

46.1 The sources

47. Running the Project, Step by Step

47.1 Prerequisites

47.2 Tests and the finding

47.3 What you should see

48. Appendix

48.1 A - Repository map

48.2 B - Glossary

CHAPTER 1

A green ETL job proves nothing

> *The pipeline running successfully catches none of 14 injected data defects. Data quality is a separate gate, and its checks are complementary -- each owns a class the others miss.*

1.1 The finding

Assay is a small retail-orders ETL -- extract three seeded source tables, join and derive an enriched table, load into SQLite -- plus a suite of data-quality checks. A harness injects a catalog of 15 realistic defects one at a time and measures which check catches which.

'The pipeline ran' catches 0 of 14 injected data defects. The expectation suite catches 8, reconciliation 7, referential 2, distribution 1; 2 slip past every check. No check flagged the 1 additive change (0 false positives).

Table 1.1 - catch counts by check (reports/summary.json)

Check	Defects caught (of 14)	What it owns
pipeline_ran (baseline)	0	nothing -- a job that loads garbage still 'succeeds'
schema	1	missing / mistyped required columns
expectations (GE-style)	8	per-row value rules -- the workhorse
referential	2	foreign keys that don't resolve
reconciliation	7	counts, sums, idempotency vs source of truth
distribution	1	numeric drift where every row is valid

Of 14 defects, 12 are caught by at least one check and 2 slip past all six. The baseline -- 'the pipeline ran' -- catches 0, because a tolerant ETL happily loads dirty data and reports success.

1.2 Complementary, not ranked

The checks are not a ranking with a single winner; each owns a class the others are blind to:

- ["**Reconciliation catches what per-row checks can't** (D11, D10, D12): a silent row drop, a non-idempotent load, a scale error.", '**Referential catches the orphan** (D03): a foreign key that points nowhere.', '**Distribution catches drift a loose range misses** (D11).', '**And some corruption escapes everything**: a valid-but-wrong value, and an encoding-mangled string.']

CHAPTER 2

Why 'the pipeline ran' is the trap

> *The baseline catches nothing on purpose -- and it is the belief most quietly held.*

2.1 Success is not quality

Every project in this series has the same shape: a trivial baseline that looks like testing and is not. For a web app it was the smoke test; for an API, the provider's own green suite; here it is the ETL job exiting zero. All three pass while the thing they guard is broken.

'The pipeline ran' catches 0 of 14 defects, because a tolerant job loads whatever it is given and reports success. This is the most common real-world failure in data engineering: a green Airflow DAG, a green dbt run, and a warehouse full of nulls and orphans. The job's exit code measures whether the code ran, not whether the data is right -- and those are different questions that need different instruments. Naming the gap between them is the whole point of the study.

CHAPTER 3

The pipeline

> *Three seeded source tables with known-clean ground truth.*

3.1 Generation

The sources -- customers, products, orders -- are generated deterministically from a fixed seed, so every run sees the same clean data and the results reproduce. The clean data satisfies every quality rule; the defect injector mutates a copy to introduce exactly one defect at a time.

Listing 2.1 - seeded source generation — pipeline/generate.py

```
def generate() -> dict:
    """Return clean source tables as {'customers': [...], 'products': [...], 'orders': [...]}."""
    rng = random.Random(SEED)

    customers = []
    for i in range(1, N_CUSTOMERS + 1):
        customers.append({
            "id": i,
            "name": f"Customer {i}",
            "email": f"customer{i}@example.com",
            "country": rng.choice(COUNTRIES),
            "signup_date": f"20{rng.randint(18, 25):02d}-{rng.randint(1,12):02d}-
{rng.randint(1,28):02d}",
        })

    products = []
    for i in range(1, N_PRODUCTS + 1):
        products.append({
            "id": i,
            "name": f"Product {i}",
            "category": rng.choice(CATEGORIES),
            "price": round(rng.uniform(5.0, 200.0), 2),
        })

    orders = []
    for i in range(1, N_ORDERS + 1):
        prod = rng.choice(products)
        orders.append({
            "id": i,
            "customer_id": rng.randint(1, N_CUSTOMERS),
            "product_id": prod["id"],
            "quantity": rng.randint(1, 20),
            "unit_price": prod["price"],           # price snapshot at order time
            "order_date": f"2026-{rng.randint(1,12):02d}-{rng.randint(1,28):02d}",
        })

    return {"customers": customers, "products": products, "orders": orders}
```

3.2 Clean data satisfies every rule

The generated clean data is, by construction, defect-free -- every order references a real customer and product, quantities and prices are in range. The generator test pins that, so the baseline the checks run against is genuinely clean and a green result on clean data is meaningful.

Listing 2.2 - the clean data is clean `tests/unit/test_generate.py`

```
def test_clean_data_shapes():
    s = generate()
    assert len(s["customers"]) == 40
    assert len(s["products"]) == 25
    assert len(s["orders"]) == 300
    # every order references a real customer and product
    cids = {c["id"] for c in s["customers"]}
    pids = {p["id"] for p in s["products"]}
    for o in s["orders"]:
        assert o["customer_id"] in cids
        assert o["product_id"] in pids
        assert 1 <= o["quantity"] <= 20
        assert o["unit_price"] > 0
```

CHAPTER 4

The source and warehouse schemas

> *Three source tables in, one enriched table out.*

4.1 The orders source

Orders reference a customer and a product and carry a quantity, a price snapshot, and a date -- the shape most defects target.

Listing SC.1 - generating orders — pipeline/generate.py

```
orders = []
for i in range(1, N_ORDERS + 1):
    prod = rng.choice(products)
    orders.append({
        "id": i,
        "customer_id": rng.randint(1, N_CUSTOMERS),
        "product_id": prod["id"],
        "quantity": rng.randint(1, 20),
        "unit_price": prod["price"],           # price snapshot at order time
        "order_date": f"2026-{rng.randint(1,12):02d}-{rng.randint(1,28):02d}",
```

4.2 The warehouse table

The enriched warehouse table is a fixed set of typed columns; the load derives the DDL from them.

Listing SC.2 - the warehouse columns — pipeline/load.py

```
COLUMNS = [
    "order_id", "customer_id", "customer_name", "country", "product_id",
    "product_name", "category", "quantity", "unit_price", "line_total",
    "order_date", "email",
]
```

Listing SC.3 - the warehouse connection + schema — pipeline/load.py

```
def connect() -> sqlite3.Connection:
    conn = sqlite3.connect(":memory:")
    conn.row_factory = sqlite3.Row
    conn.executescript(SCHEMA)
    return conn
```

CHAPTER 5

Products and customers

> *The lookup tables the orders join against.*

5.1 The catalog

Products carry a price the orders snapshot, and a category the expectation suite could constrain. Customers carry the email and country the value checks validate.

Listing G2.1 - generating products `pipeline/generate.py`

```
products = []
for i in range(1, N_PRODUCTS + 1):
    products.append({
        "id": i,
        "name": f"Product {i}",
        "category": rng.choice(CATEGORIES),
        "price": round(rng.uniform(5.0, 200.0), 2),
```

CHAPTER 6

The E in ETL: extraction hazards

> *Most data defects are born at extraction; the study injects them, but here is where they come from.*

6.1 Where bad data comes from

This project injects defects directly, but in reality they are born at extraction, and a data tester should know the usual sources:

- ['**Encoding** -- a CSV read as latin-1 instead of UTF-8 produces the mojibake defect (D14); always decode explicitly.', '**Delimiters and quoting** -- an unescaped comma or newline inside a field shifts every column right, producing type and referential defects downstream.', '**Partial / truncated files** -- a transfer that stops early is the real-world silent row drop (D10); a byte count or manifest catches it before load.', '**Schema-on-read drift** -- the source adds, drops or reorders a column and the extractor, positional rather than named, reads garbage.', '**Timezone and unit ambiguity** -- a date without a zone, or a price without a currency, is the source of the scale (D11) and silent-corruption (D13) defects.']

The lesson: validate at the boundary, as close to extraction as possible, because a defect that reaches the warehouse has already been copied into every downstream table. The checks here run post-load to measure them cleanly; a real pipeline pushes as many as it can to the edge.

CHAPTER 7

The transform

> *Pure helpers plus a mechanical join/derive that never validates or repairs -- the honest model of a real pipeline.*

7.1 Pure helpers

The decisions live in small, pure, property-tested functions: is this a valid date, a valid email, what is the line total.

Listing 3.1 - ISO-date validation — pipeline/transform.py

```
def is_iso_date(s) -> bool:
    if not isinstance(s, str) or not _DATE_RE.match(s):
        return False
    y, m, d = (int(x) for x in s.split("-"))
    if not (1 <= m <= 12 and 1 <= d <= 31):
        return False
    return True
```

Listing 3.2 - the derived field (with the injectable bug path) — pipeline/transform.py

```
def compute_line_total(quantity, unit_price, *, bug: bool = False) -> float | None:
    """quantity * unit_price, rounded. The `bug` path adds instead of multiplying
    -- the injected computed-field defect."""
    if quantity is None or unit_price is None:
        return None
    try:
        if bug:
            return round(float(quantity) + float(unit_price), 2)
        return round(float(quantity) * float(unit_price), 2)
```

7.2 The mechanical join

The transform joins orders to customers and products, casts, and derives line_total. Crucially it does NOT validate or silently repair -- injected defects flow through to the warehouse, where the checks catch them (or do not). That is the honest model: the job maps and loads whatever it is given, and quality is a separate gate.

Listing 3.3 - join, cast, derive — pipeline/transform.py

```python
def transform(sources: dict, process_bug: str | None = None) -> list[dict]:
    """Join orders to customers and products, cast, and derive line_total.
    Returns the enriched rows to be loaded. Never raises on dirty data; never
    repairs it."""
    customers = {c["id"]: c for c in sources["customers"]}
    products = {p["id"]: p for p in sources["products"]}

    rows: list[dict] = []
    for i, o in enumerate(sources["orders"]):
        if process_bug == "D10" and i % 8 == 0:
            continue                                # silent-drop bug: skip ~12% of rows

        cust = customers.get(o.get("customer_id"))
        prod = products.get(o.get("product_id"))
        rows.append({
            "order_id": o.get("id"),
            "customer_id": o.get("customer_id"),
            "customer_name": cust["name"] if cust else None,
            "country": cust["country"] if cust else None,
            "product_id": o.get("product_id"),
            "product_name": prod["name"] if prod else None,
            "category": prod["category"] if prod else None,
            "quantity": o.get("quantity"),
            "unit_price": o.get("unit_price"),
            "line_total": compute_line_total(o.get("quantity"), o.get("unit_price"),
                                             bug=(process_bug == "D09")),
            "order_date": o.get("order_date"),
            "email": cust["email"] if cust else None,
        })
    return rows
```

CHAPTER 8

The load

> *A SQLite warehouse, with replace semantics that make a re-run idempotent.*

8.1 Replace, not append

The load writes into a SQLite table. Replace semantics (delete-then-insert) make a re-run idempotent -- loading the same batch twice leaves the table unchanged. The idempotency defect skips the delete, so a re-run doubles the rows, and reconciliation catches it.

Listing 4.1 - the idempotent load — pipeline/load.py

```
def load(conn: sqlite3.Connection, rows: list[dict], *, append: bool = False) -> None:
    if not append:
        conn.execute("DELETE FROM orders_enriched")     # replace => idempotent
    placeholders = ", ".join("?" for _ in COLUMNS)
    conn.executemany(
        f"INSERT INTO orders_enriched ({', '.join(COLUMNS)}) VALUES ({placeholders})",
        [tuple(r.get(c) for c in COLUMNS) for r in rows],
    )
    conn.commit()
```

CHAPTER 9

Orchestrating the run

> *Extract, transform, load -- kept independent of the defect catalog.*

9.1 The runner

The orchestrator is deliberately thin and independent of the defect catalog: callers pass already-prepared sources and an optional process-bug flag, so tests and the harness drive it the same way. It runs the load into a caller-supplied connection, which is what lets the harness load twice into one store to judge idempotency.

Listing ER.1 - extract, transform, load — pipeline/run_etl.py

```
def run(sources: dict, process_bug: str | None = None,
        conn: sqlite3.Connection | None = None) -> tuple[list[dict], sqlite3.Connection]:
    conn = conn or loadmod.connect()
    rows = transform(sources, process_bug=process_bug)
    loadmod.load(conn, rows, append=(process_bug == "D12"))
    return loadmod.read(conn), conn
```

CHAPTER 10

The check suite

> *Six mechanisms, cheapest first -- the shape of a data-quality gate.*

10.1 The mechanisms

The suite runs six checks over the loaded warehouse and reports which flagged the data. Ordered by cost and by what they can see:

Listing 5.1 - the check suite — checks/suite.py

```
def run(ctx) -> dict:
    return {
        "pipeline_ran": not ctx.ran_ok,        # only 'catches' if the job crashed
        "schema": bool(schema.check(ctx)),
        "expectations": bool(expectations.check(ctx)),
        "referential": bool(referential.check(ctx)),
        "reconciliation": bool(reconciliation.check(ctx)),
        "distribution": bool(distribution.check(ctx)),
```

Each check reads a shared context: the loaded rows, the source tables, a second load (for idempotency), whether the job completed, and a profile of the known-good data to reconcile against.

Listing 5.2 - the known-good baseline profile — checks/context.py

```
def build_profile(clean_loaded: list[dict]) -> dict:
    prices = [r["unit_price"] for r in clean_loaded if isinstance(r["unit_price"], (int, float))]
    totals = [r["line_total"] for r in clean_loaded if isinstance(r["line_total"], (int, float))]
    return {
        "count": len(clean_loaded),
        "line_total_sum": round(sum(totals), 2),
        "unit_price_mean": statistics.fmean(prices),
        "unit_price_std": statistics.pstdev(prices),
```

CHAPTER 11

Profiling before expecting

> *You do not guess expectations -- you profile known-good data and encode what you see.*

11.1 The profile is the source of the rules

An expectation suite is not invented; it is read off a profile of known-good data. What is the range of quantity, the set of countries, the mean price, the uniqueness of order_id? The baseline profile this study builds for reconciliation and distribution is a tiny data profiler -- count, sum, mean, spread -- and in a real project the same step (df.describe(), a GE profiler, a Soda profile) is what generates the first draft of the expectations.

The discipline: profile the data you trust, encode the profile as expectations, then let the checks tell you when new data leaves the envelope. Writing expectations from imagination instead of from a profile is how suites end up both too strict (false positives on day one) and too loose (missing the defects that matter).

CHAPTER 12

Schema checks

> *The structural gate: required columns, present and typed.*

12.1 Structure only

The schema check confirms every warehouse row has the required columns, correctly typed and non-null where required. It is the structural gate -- it catches a missing or mistyped field, and is blind to whether the values themselves are sensible.

Listing 6.1 - schema checks — checks/schema.py

```
def check(ctx) -> list[str]:
    v: list[str] = []
    for r in ctx.loaded:
        for c in REQUIRED_INT:
            if r.get(c) is None:
                v.append(f"schema: {c} is null")
            elif not _is_int(r[c]):
                v.append(f"schema: {c} is not an int")
        for c in REQUIRED_NUM:
            if r.get(c) is None:
                v.append(f"schema: {c} is null")
            elif not _is_num(r[c]):
                v.append(f"schema: {c} is not numeric")
        for c in REQUIRED_STR:
            if not isinstance(r.get(c), str):
                v.append(f"schema: {c} is not a string")
    return list(dict.fromkeys(v))
```

CHAPTER 13

The expectation suite

> *Column-level value rules -- the workhorse, and its blind spot.*

13.1 Great-Expectations, hand-rolled

The expectation suite is the workhorse: not-null, unique, between, in-set, matches-regex, and a computed-field consistency check. The vocabulary mirrors Great Expectations on purpose, so the skill transfers directly to GE, Soda or dbt tests.

Listing 7.1 - the expectation suite — checks/expectations.py

```
def check(ctx) -> list[str]:
    v: list[str] = []
    rows = ctx.loaded

    # not-null
    for r in rows:
        if r.get("order_id") is None:
            v.append("expect order_id not null")
        if r.get("customer_id") is None:
            v.append("expect customer_id not null")

    # unique order_id
    ids = [r.get("order_id") for r in rows]
    if len(ids) != len(set(ids)):
        v.append("expect order_id unique")

    for r in rows:
        q = r.get("quantity")
        if not (isinstance(q, (int, float)) and 1 <= q <= 100):
            v.append("expect quantity between 1 and 100")
        up = r.get("unit_price")
        if not (isinstance(up, (int, float)) and 0.01 <= up <= 1_000_000):
            v.append("expect unit_price between 0.01 and 1e6")
        if not is_iso_date(r.get("order_date")):
            v.append("expect order_date to be a valid ISO date")
        if r.get("email") is not None and not is_valid_email(r.get("email")):
            v.append("expect email to match the email pattern")
        if r.get("country") is not None and r.get("country") not in VALID_COUNTRIES:
            v.append("expect country in the known set")
        # computed-field consistency: line_total == quantity * unit_price
        expected = compute_line_total(q, up)
        if expected is not None and r.get("line_total") is not None:
            if abs(r["line_total"] - expected) > 0.01:
                v.append("expect line_total == quantity * unit_price")

    return list(dict.fromkeys(v))
```

It catches 8 of 14 defects -- every per-row value defect. What it cannot see is anything where each row is individually valid: a silently dropped row, a scale error inside a loose range, a valid-but-wrong value. Those are the other checks' job, and the study's gaps.

CHAPTER 14

Referential integrity

> *Foreign keys must resolve -- a defect no per-row value check sees.*

14.1 Keys that point nowhere

Every foreign key in the warehouse must resolve to a real row in the source lookup tables. A per-row value check on the order cannot see that its customer_id points at a customer who does not exist; this check exists precisely to catch that orphan.

Listing 8.1 - referential integrity — checks/referential.py

```
def check(ctx) -> list[str]:
    v: list[str] = []
    customer_ids = {c["id"] for c in ctx.sources["customers"]}
    product_ids = {p["id"] for p in ctx.sources["products"]}
    for r in ctx.loaded:
        if r.get("customer_id") not in customer_ids:
            v.append("referential: order.customer_id has no matching customer")
        if r.get("product_id") not in product_ids:
            v.append("referential: order.product_id has no matching product")
    return list(dict.fromkeys(v))
```

CHAPTER 15

Reconciliation

> *Counts, sums, idempotency -- the aggregate checks that catch what per-row checks miss.*

15.1 Against the source of truth

Reconciliation compares the loaded row count and an aggregate sum against a known-good source of truth, and re-runs the load to check idempotency. These aggregate checks catch silent row drops, duplicates, scale errors and non-idempotent loads -- defects where the surviving rows are each perfectly valid, so every per-row check passes.

Listing 9.1 - reconciliation — checks/reconciliation.py

```
def check(ctx) -> list[str]:
    v: list[str] = []
    exp = ctx.profile

    if abs(len(ctx.loaded) - exp["count"]) > COUNT_TOL:
        v.append(f"reconciliation: row count {len(ctx.loaded)} != expected {exp['count']}")

    totals = [r["line_total"] for r in ctx.loaded if isinstance(r["line_total"], (int, float))]
    got = round(sum(totals), 2)
    if abs(got - exp["line_total_sum"]) > max(1.0, SUM_TOL_FRAC * abs(exp["line_total_sum"])):
        v.append(f"reconciliation: line_total sum {got} != expected {exp['line_total_sum']}")

    # idempotency: a second load into the same store must not change the row count
    if len(ctx.loaded_again) != len(ctx.loaded):
        v.append(f"reconciliation: not idempotent (re-run gives {len(ctx.loaded_again)} "
                 f"rows vs {len(ctx.loaded)})")

    return v
```

THE CLEAREST CELL IN THE STUDY

The silent-drop defect removes 12% of valid rows. Every per-row check passes -- the rows that remain are fine -- and only reconciliation's row-count check catches it. It is the canonical reason a data pipeline needs reconciliation and not just column expectations.

CHAPTER 16

Distribution checks

> *Drift monitoring: catch a shift where every row is a valid number.*

16.1 Mean and spread vs a baseline

The distribution check compares the mean and spread of a numeric column against a known-good baseline profile. It catches a scale error or drift where every individual value is still a valid number inside the range check -- the tool you reach for when monitoring for drift with no source-of-truth sum to reconcile against.

Listing 10.1 - distribution drift — checks/distribution.py

```
def check(ctx) -> list[str]:
    v: list[str] = []
    prices = [r["unit_price"] for r in ctx.loaded if isinstance(r["unit_price"], (int, float))]
    if not prices:
        return v
```

A 100x price inflation keeps every value positive and inside the loose range the expectation suite allows -- so expectations miss it -- but the mean jumps far outside its baseline band.

CHAPTER 17

Choosing thresholds

> *Every aggregate check has a tolerance, and the number is a precision/recall decision.*

17.1 Too tight cries wolf; too loose misses drift

The reconciliation sum allows 0.1% drift; the distribution mean allows 30% and the spread 60%. Those numbers are not arbitrary and they are not free: tighten them and benign fluctuation trips the check (false positives, and a gate people disable); loosen them and a real defect slips under (false negatives, like the scale error would if the band were 200%). The right value comes from the profile -- how much does this metric vary run-to-run on known-good data -- plus the cost of each error type for this dataset.

It is the same precision/recall trade this portfolio measures for detectors elsewhere, applied to data quality: a threshold is a policy, and the honest way to set it is from observed variation, not a round number. The study's loose unit_price range is deliberate -- it lets the scale error pass the per-row check so the distribution and reconciliation checks can demonstrate their value.

CHAPTER 18

When a distribution check becomes anomaly detection

> *The band here is fixed; at scale it is learned, and the check becomes a model.*

18.1 Fixed bands, then learned ones

The distribution check compares the mean and spread to a baseline with a fixed tolerance -- simple and enough to catch a 100x scale error. At scale, the tolerance is learned: a z-score against the rolling history, an interquartile-range fence for outliers, or a seasonal model that knows Monday's volume differs from Sunday's. The check becomes a small anomaly-detection model, and it inherits that world's trade-off -- too tight and it cries wolf, too loose and it misses drift, exactly the precision/recall balance this portfolio measures elsewhere.

The honest boundary: a distribution check catches a shift, not a cause, and it cannot distinguish a real business change (a genuine price rise) from a defect (cents-as-dollars). That is why it warns rather than blocks, and why a human closes the loop -- the same reason the valid-but-wrong defect needs a person, not just a threshold.

CHAPTER 19

The dimensions of data quality

> *Six standard dimensions, and the check that owns each.*

19.1 A vocabulary, mapped to checks

Data quality is usually described in dimensions. Naming them shows the check suite is not ad hoc -- each dimension has an owner:

Table DQ.1 - dimensions and their owners

Dimension	Question	Owned by
Completeness	are required values present?	schema, expectations (not-null)
Validity	are values in the right form/range?	expectations (range, regex, set)
Uniqueness	are keys unique?	expectations (unique)
Consistency	do derived values agree?	expectations (computed field)
Integrity	do foreign keys resolve?	referential
Accuracy / drift	do the numbers match reality?	reconciliation, distribution

The one dimension no automated check fully owns is accuracy at the row level -- whether a specific value is *true*. That is the silent-corruption gap, and it is why reconciliation against the source system, and ultimately human review, remain part of the picture.

CHAPTER 20

The defect catalog

> *Realistic data and process defects, injected one at a time.*

20.1 Data and process defects

Each defect is a DATA defect (mutating the source tables) or a PROCESS defect (a bug in the transform/load), plus one non-defect additive change as a false-positive probe.

Listing 11.1 - the Defect record — scoring/defects.py

```
class Defect:
    id: str
    klass: str
    kind: str          # "data" | "process" | "none"
    is_defect: bool
    desc: str
```

Listing 11.2 - injecting one data defect — scoring/defects.py

```
def inject_data(sources: dict, defect_id: str | None) -> dict:
    """Mutate `sources` in place to introduce the given DATA (or none) defect.
    Process defects are handled in run_etl and leave the data untouched."""
    if defect_id is None:
        return sources
```

CHAPTER 21

How each defect is injected

> *A few catalog branches, to show the range of realistic corruption.*

21.1 Data defects

A scale error multiplies every price -- the classic 'cents loaded as dollars'. A silent corruption swaps a country for a different valid one -- individually plausible, wrong.

Listing IJ.1 - the scale-error defect — scoring/defects.py

```
elif defect_id == "D11":
    for o in orders:
        o["unit_price"] = round(o["unit_price"] * 100, 2)
```

Listing IJ.2 - the silent-corruption defect — scoring/defects.py

```
elif defect_id == "D13":
    # valid -> different-but-valid country: individually plausible, wrong
    for c in customers[:10]:
        c["country"] = "FR" if c["country"] != "FR" else "US"
```

21.2 A process defect

The silent-drop defect lives in the transform: an over-aggressive filter skips about 12% of valid rows. The rows that survive are all fine, which is why only reconciliation catches it.

Listing IJ.3 - the silent-drop process defect — pipeline/transform.py

```
if process_bug == "D10" and i % 8 == 0:
    continue                              # silent-drop bug: skip ~12% of rows
```

CHAPTER 22

The scoring harness

> *Inject, run the ETL twice, run every check, record.*

22.1 One defect, one run

For each defect the harness builds the injected sources, runs the ETL twice into one warehouse (so idempotency can be judged), and runs every check. It is in-process and seeded, so the catch matrix is byte-reproducible.

Listing 12.1 - scoring one defect — scoring/score.py

```
def run_defect(defect_id, profile) -> Context:
    sources = inject_data(generate(), defect_id)
    pbug = is_process(defect_id)
    ran_ok = True
    try:
        conn = connect()
        loaded1, _ = run(sources, pbug, conn)
        loaded2, _ = run(sources, pbug, conn)     # second load into the same store
    except Exception:
        ran_ok, loaded1, loaded2 = False, [], []
    return Context(loaded=loaded1, loaded_again=loaded2, sources=sources,
                   ran_ok=ran_ok, profile=profile)
```

CHAPTER 23

Turning the matrix into the finding

> *The complementarity numbers are computed from the matrix, not asserted.*

23.1 Who catches what the others miss

The headline claims -- reconciliation catches what expectations miss, referential catches the orphan -- are simple set logic over the matrix, so they cannot disagree with the per-defect table.

Listing AG.1 - who catches what the others miss — scoring/score.py

```
recon_only = [d for d in defects
              if caught_by(d, "reconciliation") and not caught_by(d, "expectations")]
dist_vs_exp = [d for d in defects
               if caught_by(d, "distribution") and not caught_by(d, "expectations")]
ref_only = [d for d in defects
            if caught_by(d, "referential") and not caught_by(d, "expectations")]
```

CHAPTER 24

Results

> *The full matrix, who catches what the others miss, and the gaps.*

24.1 The catch matrix

Table 13.1 - every defect, every check (reports/catch_matrix.json)

Defect	Class	ran	schema	expect	ref	recon	dist	note
D01	null-required	·	✓	✓	✓	·	·	
D02	duplicate-key	·	·	✓	·	✓	·	
D03	referential	·	·	·	✓	·	·	
D04	out-of-range	·	·	✓	·	✓	·	
D05	out-of-range	·	·	✓	·	✓	·	
D06	type-format	·	·	✓	·	·	·	
D07	format-regex	·	·	✓	·	·	·	
D08	domain-set	·	·	✓	·	·	·	
D09	computed-field	·	·	✓	·	✓	·	
D10	silent-drop	·	·	·	·	✓	·	
D11	scale-error	·	·	·	·	✓	✓	
D12	idempotency	·	·	·	·	✓	·	
D13	silent-corruption	·	·	·	·	·	·	missed
D14	encoding	·	·	·	·	·	·	missed
D15	additive	·	·	·	·	·	·	

24.2 By defect class

Table 13.2 - catch counts by defect class

Class	n	ran	schema	expect	ref	recon	dist
additive	1	0	0	0	0	0	0
computed-field	1	0	0	1	0	1	0
domain-set	1	0	0	1	0	0	0
duplicate-key	1	0	0	1	0	1	0
encoding	1	0	0	0	0	0	0
format-regex	1	0	0	1	0	0	0
idempotency	1	0	0	0	0	1	0
null-required	1	0	1	1	1	0	0
out-of-range	2	0	0	2	0	2	0
referential	1	0	0	0	1	0	0
scale-error	1	0	0	0	0	1	1
silent-corruption	1	0	0	0	0	0	0
silent-drop	1	0	0	0	0	1	0
type-format	1	0	0	1	0	0	0

24.3 The gaps

2 defects slip past every check -- the finding a green pipeline hides:

Table 13.3 - defects no check caught

Defect	Class	Why all checks miss it
D13	silent-corruption	some customer countries are changed to a DIFFERENT valid country.
D14	encoding	some customer names are mojibake from a bad encoding.

And no check flagged the additive change (0 false positives) -- adding unreferenced products to the catalog is not a defect, and nothing treated it as one.

CHAPTER 25

Tracing four defects

> *One only reconciliation catches, one only referential, one distribution, one nothing.*

25.1 D10 silent drop: only reconciliation

The transform drops 12% of rows. Every per-row check passes -- the survivors are valid -- so expectations, schema and referential all miss it (expectations caught: ·). Only reconciliation's row-count check, comparing to the known-good count, catches it (caught: ✓). This is the canonical argument for reconciliation.

25.2 D03 orphan: only referential

An order references a customer that does not exist. The value is a normal integer, so expectations pass (·); the count and sum are unchanged, so reconciliation passes (·). Only the referential check, resolving the key against the customers table, catches it (✓).

25.3 D11 scale: distribution (and reconciliation), not expectations

Every price is inflated 100x. Each value stays a positive number inside the loose range, so the expectation suite misses it (·). The aggregate sum and the mean both blow past their baselines, so reconciliation (✓) and distribution (✓) catch it. This is why a per-row range is not a drift check.

25.4 D13 valid-but-wrong: nothing

A customer's country is changed to a different real country. It is a valid string, in the known set, referentially irrelevant, and does not touch any aggregate -- so all six checks pass (caught by any: no). Only a row-level reconciliation against the source system, or human review, would catch it. Naming that is the honest close: automated data-quality checks bound the damage, they do not guarantee truth.

CHAPTER 26

Property-based testing of the transforms

> *Hypothesis checks the pure helpers over a wide input space, not a few examples.*

26.1 Properties, not examples

The pure transform helpers are tested with Hypothesis: instead of a handful of hand-picked inputs, it checks a property over a wide generated space. The line-total property is simply that the output is the product, rounded -- for any quantity and price.

Listing P.1 - a property test — tests/unit/test_transform.py

```
@given(st.integers(min_value=0, max_value=100000),
       st.floats(min_value=0, max_value=1e6, allow_nan=False, allow_infinity=False))
def test_line_total_is_product_rounded(q, p):
    assert compute_line_total(q, p) == round(q * p, 2)
```

This is the instrument the previous project (Accord) named for its gaps: a property test forces the rare input a fixed example never supplies -- an impossible date, a huge quantity, a hostile string.

CHAPTER 27

Testing the checks

> *A check that always fires, or never does, would make the study meaningless -- so the checks are pinned too.*

27.1 Meta-tests

Before the checks can be trusted to score defects, they are tested themselves: they must pass on clean data, and each must catch the defect class it owns. The sharpest meta-test pins the reconciliation-only cell -- the silent drop that per-row checks are blind to.

Listing M.1 - pinning the reconciliation-only cell — tests/checks/test_checks.py

```
def test_reconciliation_uniquely_catches_silent_drop():
    prof = _profile()
    res = suite.run(run_defect("D10", prof))
    assert res["reconciliation"] and not res["expectations"]       # per-row checks are blind
```

CHAPTER 28

More of the test suite

> *Determinism, date properties, and the false-positive meta-test.*

28.1 Determinism and date properties

Listing T2.1 - the generator is deterministic — tests/unit/test_generate.py

```
def test_generation_is_deterministic():
    a = generate()
    b = generate()
    assert a == b
```

Listing T2.2 - every valid date is accepted — tests/unit/test_transform.py

```
@given(st.integers(1900, 2100), st.integers(1, 12), st.integers(1, 28))
def test_valid_iso_dates_accepted(y, m, d):
    assert is_iso_date(f"{y:04d}-{m:02d}-{d:02d}")
```

28.2 Clean passes; additive is not flagged

Listing T2.3 - clean data passes every check — tests/checks/test_checks.py

```
def test_clean_data_passes_every_check():
    prof = _profile()
    assert suite.run(run_defect(None, prof)) == {
        "pipeline_ran": False, "schema": False, "expectations": False,
        "referential": False, "reconciliation": False, "distribution": False,
```

Listing T2.4 - the additive change is no false positive — tests/checks/test_checks.py

```
def test_additive_change_is_no_false_positive():
    prof = _profile()
    assert not any(suite.run(run_defect("D15", prof)).values())
```

CHAPTER 29

Pinning the remaining cells

> *The referential-only and distribution-only cells, as meta-tests.*

29.1 Each unique cell is a test

The disjoint cells -- the whole point of the study -- are pinned as meta-tests, so a change that blurred them would fail the build.

Listing T3.1 - referential catches the orphan — tests/checks/test_checks.py

```
def test_referential_catches_orphan():
    prof = _profile()
    assert suite.run(run_defect("D03", prof))["referential"]
```

Listing T3.2 - distribution catches what the loose range misses — tests/checks/test_checks.py

```
def test_distribution_catches_scale_error_expectations_miss():
    prof = _profile()
    res = suite.run(run_defect("D11", prof))
    assert res["distribution"] and not res["expectations"]        # loose range misses it
```

Listing T3.3 - malformed emails are rejected — tests/unit/test_transform.py

```
@given(st.sampled_from(["", "no-at", "a@b", "a b@c.com", None, 5]))
def test_invalid_emails_rejected(s):
    assert not is_valid_email(s)
```

CHAPTER 30

Precision: not flagging the safe change

> *A check that cries wolf on a benign change is as useless as one that misses defects.*

30.1 The additive probe

The catalog includes one non-defect: new products added to the catalog that no order references. A good check set must NOT flag it, and none did (0 false positives). Referential integrity earns this by checking orders against products (the new products are simply unreferenced); the aggregate and per-row checks operate on orders, which are unchanged.

Measuring false positives alongside catches is what makes a check safe to gate a pipeline on. A data-quality gate that blocks a release every time the catalog grows will be disabled within a week -- and then it protects nothing.

CHAPTER 31

Design decisions

> *Each load-bearing choice, and the alternative it beat.*

31.1 Why a mechanical transform

WHY: THE ETL MAPS AND LOADS; QUALITY IS A SEPARATE GATE

If the transform validated and repaired, injected defects would never reach the warehouse and there would be nothing to measure. That is not a contrivance -- it is how most real pipelines work: the job is mechanical, and data quality is a distinct set of checks run against the result. Separating them is what makes 'the pipeline ran' a meaningfully weak baseline.

31.2 Why hand-rolled checks, not Great Expectations

WHY: UNDERSTAND THE EXPECTATION BEFORE WIELDING THE TOOL

Hand-rolling a not-null or a between check is trivial; the value is that you then know exactly what each expectation does and where it is blind. A green GE dashboard is only trustworthy if you know it does not check reconciliation or row-level accuracy. The checks here map one-to-one onto GE, dbt and Soda, so the understanding transfers.

31.3 Why the standard library only

WHY: REPRODUCIBLE AND PORTABLE FOREVER

The pipeline and checks use only sqlite3, statistics, csv and re. No pandas, no cloud SDK -- so the whole thing runs anywhere with a Python interpreter, for free, byte-for-byte reproducibly, years from now. The concepts, not a particular library's API, are the transferable skill.

CHAPTER 32

Where the checks run: the quality gate

> *Pre-load or post-load, fail-closed or fail-open -- the check is only half the design; when and how it runs is the other half.*

32.1 A check is a decision, not just an assertion

Running a check is easy; deciding what to do when it fails is the engineering. Three axes a real gate must settle:

- ['**Where** -- validate a staging table before it is promoted to the warehouse (catch bad data before anyone queries it), or validate after load (simpler, but consumers may already have read it). This project checks post-load; a fail-closed pipeline checks a staging load first.', '**Fail-closed vs fail-open** -- does a failed check block the release (fail-closed: safest, but a flaky check halts the business) or raise an alert while the data flows (fail-open: keeps moving, risks bad data downstream)? The right answer differs per check and per consumer.', '**Blocking severity** -- a referential orphan might block, while a distribution drift only warns. Not every violation deserves the same response.']

The measurement in this book is deliberately about detection -- does a check fire -- because detection is the prerequisite. But a check that fires into a log nobody reads is a check that does nothing, so the gate design is where detection becomes protection.

CHAPTER 33

Severity, and quarantine

> *Not every bad row should stop the pipeline; some should be set aside.*

33.1 Reject, warn, or quarantine

A binary pass/fail is coarse. Mature pipelines route bad data three ways: **reject** the whole batch (a schema break -- nothing usable), **warn** and continue (a distribution drift worth a human's attention), or **quarantine** the offending rows (the 15 orders with a null customer_id go to a side table while the other 285 load). Quarantine is often the right answer for row-level defects: it preserves the good data and makes the bad data visible and fixable, instead of failing the whole load or silently loading garbage.

The checks here identify the bad rows (the expectation and referential checks report which rows and which rule), which is exactly what a quarantine step needs. Detection is the input to the routing decision; the study measures the input, and names routing as the production layer on top.

CHAPTER 34

From a red check to a green pipeline

> *Detection starts the work; triage and root cause finish it.*

34.1 What you do when a check fires

A red check is the beginning, not the end. The triage a data engineer runs, and how this project's checks support it:

1. ["**Localise** -- which rows, which column? The expectation and referential checks report the specific rule and the offending rows, not just 'the batch is bad'.", "**Classify** -- is it a data defect (bad source) or a process defect (bad transform)? The catalog's data/process split mirrors the two root-cause families.", '**Trace** -- follow the source keys upstream (lineage) to the extraction or the producing system.', '**Decide** -- reject, quarantine, or accept-and-alert, per the gate policy.', '**Prevent** -- push the check upstream (a data contract) so the same defect fails at production, not at load.']

A check that only says 'something is wrong' forces all of this to be manual. A check that says which rows, which rule, and which source key is the difference between a five-minute fix and an afternoon of SQL archaeology -- which is why the checks here return messages, not just booleans.

CHAPTER 35

The warehouse, and reconciliation as SQL

> *The load is SQLite; the aggregate checks are the COUNT and SUM a data engineer would write in SQL.*

35.1 Read-back and aggregates

The warehouse is a real (in-memory) SQLite table, read back after load. The reconciliation check's row count and aggregate sum are the Python expression of the SQL a data engineer writes to reconcile a load: `SELECT COUNT(*), SUM(line_total) FROM orders_enriched` compared against the source-system totals.

Listing SQ.1 - reading the warehouse — pipeline/load.py

```
def read(conn: sqlite3.Connection) -> list[dict]:
    return [dict(r) for r in conn.execute("SELECT * FROM orders_enriched ORDER BY rowid")]
```

In dbt this reconciliation is a singular test; in a pure-SQL stack it is a scheduled query with a threshold. The concept -- count and sum the target, compare to the source of truth -- is identical, and it is the check that catches the silent drop.

Listing SQ.2 - the warehouse DDL, derived from the columns — pipeline/load.py

```
SCHEMA = f"""
CREATE TABLE IF NOT EXISTS orders_enriched (
    {', '.join(c + ' ' + ('INTEGER' if c in ('order_id','customer_id','product_id','quantity') else
'REAL' if c in ('unit_price','line_total') else 'TEXT') for c in COLUMNS)}
);
"""
```

CHAPTER 36

Checks at scale

> *On a billion rows, how you run a check matters as much as the check.*

36.1 Sampling, push-down, and cost

The checks here scan a few hundred rows in memory. On a billion-row table the same checks must change shape or they become the most expensive query in the warehouse:

- ['**Push the check into SQL** -- a not-null or uniqueness check is a `COUNT(*) WHERE ... IS NULL` or a `GROUP BY ... HAVING COUNT(*) > 1`, run by the warehouse, not pulled into Python.', '**Sample for distribution** -- a mean or a quantile can be estimated from a sample; a uniqueness check cannot (one duplicate in a billion is invisible to a 1% sample).', '**Incremental checks** -- validate only the new partition, not the whole table, every run.']

Which checks can be sampled and which must be exhaustive is itself a design decision: aggregate and distribution checks tolerate sampling; uniqueness, referential integrity and reconciliation generally do not. Knowing that distinction is the difference between a data-quality suite that scales and one that gets turned off for being too slow.

CHAPTER 37

Data observability

> *The five pillars, and which this study exercises.*

37.1 The five pillars

Data observability is usually framed as five pillars, and the check suite touches most of them:

Table OB.1 - observability pillars vs the checks

Pillar	Meaning	In this study
Freshness	is the data recent?	not modelled (batch, no time axis)
Volume	is the row count normal?	reconciliation (count)
Schema	did the shape change?	schema check
Distribution	are the values normal?	distribution + expectations
Lineage	where did it come from / go?	referential (partial)

The gap -- freshness -- is a reminder that a batch study omits the time axis entirely. In production, 'the data is a day stale' is a quality failure as real as a null, and it needs its own check.

CHAPTER 38

From here to Great Expectations, dbt and Soda

> *The hand-rolled checks map one-to-one onto the real tools.*

38.1 The same concepts, at scale

Every check here is a small, readable version of what a production data-quality tool does, and the skill transfers directly:

Table T.1 - hand-rolled to production

This project	The real tool
the expectation suite	Great Expectations expectation suites; Soda checks; dbt test / dbt-expectations
reconciliation (counts, sums)	dbt tests + source freshness; reconciliation frameworks; audit-log tables
referential integrity	dbt relationships tests; database FKs
distribution drift	GE distributional expectations; Soda anomaly detection; Evidently / monitoring
the defect catalog + harness	mutation-style data testing; data contract CI

Hand-rolling them first is the point: you understand what an expectation IS, why reconciliation is a separate concern, and where each is blind -- which is exactly what lets you wield the heavy tools well rather than trusting a green dashboard.

CHAPTER 39

Data contracts

> *The upstream fix: agree the shape and semantics before the data moves.*

39.1 Shift the guarantee upstream

Every defect here is caught (or missed) downstream, after bad data has already been produced. The upstream analogue of the previous project's API contract is a **data contract**: the producer of the customers/orders data commits to a schema and to semantic rules (country is a valid ISO code; unit_price is in dollars; order_id is unique), and that contract is enforced at the boundary -- in CI on the producing service, or as a schema-registry check on the event stream.

A data contract would turn several of this study's defects into produce-time failures rather than load-time catches -- and could even express the semantic rules (a unit, an enum) that a value matcher needs, closing part of the silent-corruption gap. The checks here are the safety net; the contract is the discipline that means the net catches less because less bad data is produced.

CHAPTER 40

Quality in production

> *The checks that run after deploy: freshness, volume, and drift.*

40.1 From batch checks to monitors

The same checks that gate a batch become monitors in production. Three that a real deployment adds on top of this study's six:

- ['**Freshness** -- has the data updated within its SLA? (a timeliness check the batch study does not model)', "**Volume anomaly** -- is today's row count within the normal band for this day of week? (reconciliation's count check, made adaptive)", "**Drift over time** -- is a column's distribution moving across runs, not just against one baseline? (distribution, tracked)"]

The bridge is the same as in the previous project: a check is a description of what good data looks like, usable both to gate a deploy and to watch production. The silent-corruption gap is exactly where production monitoring (comparing to the operational source system) earns its keep, because pre-load checks structurally cannot see it.

CHAPTER 41

Incremental loads and late data

> *The hazards a full-refresh study does not model.*

41.1 Why full-refresh hides problems

This pipeline is a full refresh: every run rebuilds the whole warehouse from the whole source, which is why replace-semantics make idempotency trivial. Real pipelines are usually incremental -- they load only what changed since the last run -- and that opens hazards this study does not model: a late-arriving record for a window already loaded, a watermark that skips rows, a backfill that double-counts, a deletion in the source that the incremental load never sees.

Each of those is a reconciliation problem at heart -- the loaded total drifts from the source total over time -- which is why reconciliation against the source system, not just per-row checks, is the check that scales to incremental pipelines. The full-refresh model here makes the check comparison clean; the incremental world makes reconciliation even more essential, not less.

CHAPTER 42

Lineage and provenance

> *Knowing where a value came from is what turns a caught defect into a fixed one.*

42.1 Detection is not enough without lineage

A check that fires tells you the warehouse is wrong; lineage tells you why. The enriched row carries its source keys (order_id, customer_id, product_id), so a failed expectation can be traced back to the exact source record and the join that produced it. Without that, a distribution drift is an alert with nowhere to go.

At scale this is what tools like dbt's lineage graph, OpenLineage and column-level lineage provide: given a bad value, the set of upstream models and sources that could have caused it. The study's referential check is the smallest version -- it does not just say 'this key is bad', it says 'this order points at a customer that is not in the customers source' -- which is a one-hop lineage claim.

CHAPTER 43

A single data-quality score, and why to distrust it

> *Rolling the checks into one number is tempting and lossy.*

43.1 The percentage that hides the gap

It is tempting to roll the matrix into one figure -- 'the pipeline is 86% quality-passing' -- and dashboards love it. But a single score hides exactly what this study surfaces: which class of defect is uncaught. A dataset can score 95% while every one of its silent corruptions goes undetected, because those defects are not in the denominator -- no check tests for them, so they never fail and never lower the score.

A quality score is fine as a trend to watch, dangerous as a gate to trust. The honest artifact is the matrix, not the average: it names what is checked, what is caught, and -- most importantly -- what is not checked at all. The same lesson as coverage-versus-correctness in the web project: a high number measures what you tested, not what is true.

CHAPTER 44

What this does not establish

> *The honest boundary around a curated, single-dataset study.*

44.1 A curated catalog

The 14 defects are hand-picked to span classes cleanly, one representative each. The catch counts are illustrative of each check's reach, not an estimate of any dataset's defect rate. And the gaps are the gaps of THIS check set: a charset expectation would catch the encoding defect, and row-level reconciliation against the source system would catch the valid-but-wrong value. What is robust is the structure -- expectations own per-row values, reconciliation owns aggregates, referential owns keys, distribution owns drift, and some corruption escapes automated checking.

44.2 'Caught' means one check fired

A check catches a defect if it produced at least one violation. The study does not grade how precisely the violation localises the bad rows, or how good the message is. And the pipeline here is small and in-memory; a real warehouse adds partitioning, incremental loads and late-arriving data, each with its own quality hazards the six checks here do not model.

CHAPTER 45

The finding, restated

> *One table, and the boundary around it.*

45.1 Everything in one place

Table RC.1 - the whole project, from reports/summary.json

Question	Answer (reports/summary.json)
Injected data defects	14
Caught by 'the pipeline ran'	0
Caught by schema	1
Caught by expectations	8
Caught by referential	2
Caught by reconciliation	7
Caught by distribution	1
Caught by at least one check	12
Slip past every check	2
False positives on the additive change	0

45.2 The paragraph

'The pipeline ran' catches 0 of 14 injected data defects. The expectation suite catches 8, reconciliation 7, referential 2, distribution 1; 2 slip past every check. No check flagged the 1 additive change (0 false positives). A green pipeline is not clean data; each check owns a class the others miss -- expectations the per-row values, reconciliation the aggregates and idempotency, referential the keys, distribution the drift -- and a valid-but-wrong value escapes them all. Data quality is layered, and the honest edge is that automated checks bound the damage rather than guarantee truth, which is why reconciliation against the source system and a data contract upstream complete the picture.

CHAPTER 46

Where to read further

> *Data-quality dimensions, the tools, and the testing ideas.*

46.1 The sources

The vocabulary of data-quality dimensions -- completeness, validity, consistency, uniqueness, timeliness, accuracy -- is standard; the DAMA-DMBOK is the reference. For the tools, the Great Expectations docs (expectation suites, checkpoints), the dbt testing guide (generic + singular tests, dbt-utils / dbt-expectations) and Soda's checks language are the practical reads. The idea that reconciliation is a first-class pipeline concern -- not a special case of column tests -- is made well in Kleppmann's *Designing Data-Intensive Applications* (batch chapter) and in the 'data contracts' literature. Hypothesis's docs cover property-based testing; its 'find the edge case you didn't think of' philosophy is exactly right for validating transform functions.

CHAPTER 47

Running the Project, Step by Step

> *From a clone to the catch matrix, on Python's standard library.*

47.1 Prerequisites

You need **Python 3.12+**. The pipeline and checks use only the standard library; the three dependencies are for the tests and the measurement.

Toolcheck

```
python --version
```

47.2 Tests and the finding

Everything measured

```
python -m venv .venv
.venv\Scripts\activate            # Windows
source .venv/bin/activate           # macOS / Linux
pip install -r requirements.txt

python -m pytest tests              # 18 tests
python scoring/score.py             # the finding -> reports/*.json
```

47.3 What you should see

`18 passed`; `score.py` prints a per-defect matrix and writes `reports/catch_matrix.json` + `summary.json`, byte-identical on re-run.

CHAPTER 48

Appendix

> *Repository map and glossary.*

48.1 A - Repository map

Where everything lives

Path	Purpose
`pipeline/generate.py`	seeded source generation
`pipeline/transform.py`	pure helpers + the join/derive
`pipeline/load.py`	the SQLite warehouse load
`checks/expectations.py`	the GE-style expectation suite
`checks/reconciliation.py`	counts, sums, idempotency
`checks/referential.py`	foreign-key integrity
`checks/distribution.py`	numeric drift
`scoring/defects.py`	the defect catalog + injector
`scoring/score.py`	the fault-injection harness
`reports/*.json`	the source of every number here

48.2 B - Glossary

Terms used in this book

Term	Meaning
Data-quality check	an assertion about data, separate from 'did the job run'
Expectation	a column-level value rule (GE vocabulary)
Reconciliation	comparing counts/sums against a source of truth
Referential integrity	every foreign key resolves to a real row
Idempotency	re-running the load leaves the result unchanged
Distribution drift	a numeric column's mean/spread moving from baseline
Silent corruption	a wrong value that is still individually valid
Property-based testing	checking a property over generated inputs (Hypothesis)
Fault injection	deliberately introducing a defect to see if a check notices